200.5.0. EFU SYSTEM SOFTWARE SPECIFICATION v1.0

Classification: Digital Backbone – Master Integration Document
Main Function: Unified software architecture for EFU system implementation

Version: 1.0
Date: 2026-03-06
Author: István Simor

Related Documents:
• 205.1–205.4 Technical Specifications
• 900.5 Dashboard
• 104.60 CEWS

EXECUTIVE SUMMARY

The EFU System Software Specification defines the digital architecture required to operationalize the EFU civilizational monitoring framework. The platform integrates more than one hundred analytical modules into a scalable infrastructure capable of real-time monitoring, scenario simulation, and early warning detection.

The architecture is based on a four-layer model:

• API Layer
• Data Layer
• Application Logic Layer
• User Interface Layer

This document defines the data model, API interfaces, computational modules, dashboard integration, and deployment roadmap necessary to implement the EFU platform.

---

1. PURPOSE AND SCOPE

The **200.5 EFU System Software Specification** is the master integration document that unifies all technical components of the EFU digital infrastructure.

Key Question:
How do we integrate 100+ modules (104.x licences, 600.x Antiflux, 700.x Regenerative) into a functioning software system?

Answer:
Through a modular, interoperable architecture with:

• Standardized data model (205.2)
• JSON validation layer (205.3)
• RESTful API (205.4)
• User interfaces (900.5 Dashboard, calculators, scenario tools)

---

2. SYSTEM ARCHITECTURE OVERVIEW

2.1 Four-Layer Architecture

Layer 4 – USER INTERFACE

• 900.5 Dashboard (CII, VKI, SII, CEWS widgets)
• 104.60 CEWS Interface (real-time monitoring)
• Scenario Calculator (what-if analysis)
• Public Web integration

↓

Layer 3 – APPLICATION LOGIC

• 104.x Licence Calculators (Energy, Water, PFAS, etc.)
• 600.x Antiflux Detectors (parasite scoring)
• 700.x Regenerative Evaluators (symbiotic scoring)
• 104.60 CEWS Engine (dX/dt, d^2X/dt^2 calculations)
• 900.1 CDS Diagnostics (CII, VKI, SII computation)

↓

Layer 2 – DATA LAYER (205.2, 205.3)

• Data Model (EFU units, fluxes, entities)
• JSON Schema Validation
• Time-series storage (dX/dt historical data)
• Relational database (PostgreSQL recommended)

↓

Layer 1 – API LAYER (205.4)

• RESTful API endpoints (GET, POST, PUT, DELETE)
• Authentication (API keys, OAuth)

• Rate limiting
• Documentation (Swagger / OpenAPI)

2.2 Core Components

1. EFU Calculator Engine

Input:
Raw data (energy consumption, population, water use)

Process:
Convert to EFU units (20 kg/day/person reference)

Output:
EFU values, SS indices, W_irrev scores

---

2. CEWS Temporal Engine (104.60)

Input:
Time-series data (CII, VKI, SII)

Process:
Calculate temporal derivatives:

• dX/dt (velocity)
• d^2X/dt^2 (acceleration)

Output:
Alert levels (Green / Yellow / Red)

---

3. Dashboard Visualization (900.5)

Input:
EFU data, CEWS alerts, module outputs

Process:
Real-time rendering (charts, widgets, maps)

Output:
Interactive web interface

---

4. Scenario Simulator

Input:
User-defined parameters
Example: *"What if Nigeria's TFR drops to 3.0?"*

Process:
Run relevant modules (104.39, 104.44-46)

Output:
Projected values:

• SS_total
• CII
• VKI
• SII

for years **2030 / 2050 / 2100**

---

3. DATA MODEL INTEGRATION (205.2)

3.1 Core Entities

Example JSON structure:

```
{
  "entity": {
    "id": "HU-2024",
    "name": "Hungary",
    "type": "nation",
    "population": 9700000,
    "area_km2": 93030
  },
  "flux": {
    "EFU_H": 60000000,
    "EFU_M": 45000000,
    "EFU_S": 38000000
  },
  "indices": {
    "CII": 0.76,
    "VKI": 0.68,
    "SII": 0.42,
    "SS_total": -8.5,
    "W_irrev_avg": 420
  },
  "temporal": {
    "dCII_dt": -0.01,
    "d2CII_dt2": 0.001,
    "dVKI_dt": -0.03,
    "d2VKI_dt2": -0.005
  }
```

```
}
```

3.2 Relational Schema (PostgreSQL)

Entities Table

```
CREATE TABLE entities (
    id SERIAL PRIMARY KEY,
    external_id VARCHAR(50) UNIQUE,
    name VARCHAR(255),
    type VARCHAR(50),
    population BIGINT,
    area_km2 DECIMAL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Flux Data Table

```
CREATE TABLE flux_data (
    id SERIAL PRIMARY KEY,
    entity_id INT REFERENCES entities(id),
    timestamp TIMESTAMP,
    EFU_H DECIMAL,
    EFU_M DECIMAL,
    EFU_S DECIMAL,
    source VARCHAR(255)
);
```

Indices Table

```
CREATE TABLE indices (
    id SERIAL PRIMARY KEY,
    entity_id INT REFERENCES entities(id),
    timestamp TIMESTAMP,
    CII DECIMAL,
    VKI DECIMAL,
    SII DECIMAL,
    SS_total DECIMAL,
    W_irrev_avg DECIMAL
);
```

Temporal Derivatives Table

```
CREATE TABLE temporal_derivatives (
    id SERIAL PRIMARY KEY,
    entity_id INT REFERENCES entities(id),
    timestamp TIMESTAMP,
    metric VARCHAR(50),
    dX_dt DECIMAL,
    d2X_dt2 DECIMAL
);
```

Module Outputs Table

```
CREATE TABLE module_outputs (
    id SERIAL PRIMARY KEY,
    entity_id INT REFERENCES entities(id),
    module_id VARCHAR(50),
    timestamp TIMESTAMP,
    output_data JSONB
);
```

---

4. API SPECIFICATION (205.4)

Base URL

```
https://api.efu-system.org/v1
```

Authentication: Bearer Token (API Key)
Rate Limit: 1000 requests per hour per key

4.1 Entity Management

Endpoints:

```
GET /entities
GET /entities/{id}
POST /entities
PUT /entities/{id}
DELETE /entities/{id}
```

Example request:

```
GET /entities/HU-2024
Authorization: Bearer YOUR_API_KEY
```

Example response:

```
{
  "id": "HU-2024",
  "name": "Hungary",
  "type": "nation",
  "population": 9700000,
  "area_km2": 93030,
  "indices": {
    "CII": 0.76,
    "VKI": 0.68,
    "SII": 0.42
  }
}
```

4.2 Flux Data

```
GET /flux/{entity_id}
POST /flux/{entity_id}
```

Example request:

```
POST /flux/HU-2024
Content-Type: application/json
{
  "timestamp": "2024-03-06T12:00:00Z",
  "EFU_H": 60000000,
  "EFU_M": 45000000,
  "EFU_S": 38000000
}
```

4.3 CEWS Alerts

Endpoints:

```
GET /cews/{entity_id}
GET /cews/{entity_id}/alerts
```

Example response:

```
{
  "entity_id": "HU-2024",
  "status": "yellow",
  "metrics": {
    "CII": {
      "value": 0.76,
      "dX_dt": -0.01,
      "d2X_dt2": 0.001,
      "zone": "green"
    }
  }
}
```

4.4 Module Calculations

Endpoint:

```
POST /modules/{module_id}/calculate
```

Example (Module 104.39):

```
POST /modules/104.39/calculate
```

Input:

```
{
  "entity_id": "Niger-2024",
```

```
  "N_population": 27000000,
  "TFR": 6.7,
  "projection_year": 2050
}
```

Output:

```
{
  "module_id": "104.39",
  "entity_id": "Niger-2024",
  "result": {
    "N_pop_2050": 66000000,
    "dN_pop_dt": 750000,
    "SS_soil_2050": -18.9,
    "cews_zone": "yellow"
  }
}
```

---

5. DASHBOARD INTEGRATION (900.5)

5.1 Technology Stack

Frontend framework: **React.js / Next.js**

Visualization libraries:

• D3.js
• Chart.js
• Recharts

Real-time updates:

• WebSocket

State management:

• Redux or Zustand

API Client:

• Axios or Fetch

5.2 Dashboard Widgets

Entity Overview

Displays:

• CII, VKI, SII gauges
• SS_total color indicator
• Population and basic statistics

CEWS Temporal Widget (104.60)

Displays:

• dCII/dt, dVKI/dt, dSII/dt
• d^2CII/dt^2, d^2VKI/dt^2, d^2SII/dt^2
• Zone status (Green / Yellow / Red)
• Historical trends

Module Outputs

Examples:

• 104.39 Biosphere population projections
• 104.44–46 planetary flux trends
• 600.52 PFAS metrics
• 700.x regenerative scores

Scenario Simulator

Features:

• Parameter sliders
• What-if projections
• Baseline vs scenario comparison charts

---

6. SCENARIO CALCULATOR

6.1 Use Cases

Demographic Scenario

"What if Nigeria’s TFR drops to 3.0 by 2030?"

→ Run module 104.39

Energy Scenario

"What if Hungary doubles renewable energy by 2030?"

→ Run module 104.48

Antiflux Scenario

"What if PFAS production stops globally in 2025?"

→ Run module 600.52

Regenerative Scenario

"What if 30% of agriculture becomes regenerative?"

→ Run module 700.1

---

7. DEPLOYMENT AND INFRASTRUCTURE

Backend

Language:

Python (FastAPI or Django REST)

Database:

PostgreSQL

Cache:

Redis

Task queue:

Celery

Frontend

Framework:

React.js / Next.js

Visualization:

D3.js / Recharts

Real-time updates:

WebSocket

Deployment

Containers:

Docker

Orchestration:

Kubernetes (optional)

Cloud providers:

• AWS
• Google Cloud
• DigitalOcean

CI/CD:

• GitHub Actions
• GitLab CI

---

8. IMPLEMENTATION ROADMAP

Phase 1 – MVP (3–6 months)

Core API
Basic dashboard
2 module integrations

Modules:

• 104.39 Biosphere
• 104.60 CEWS

Phase 2 – Full System (6–12 months)

Full API
Real-time alerts
Scenario calculator
10+ modules integrated

Phase 3 – Scaling (12–24 months)

• Multi-entity analysis
• Public API
• Mobile application
• AI integration

---

9. FINAL STATEMENT

The **200.5 EFU System Software Specification** is the digital backbone that transforms the EFU from theory into an operational system.

It integrates **100+ modules (104.x, 600.x, 700.x)** into a scalable and auditable platform.

Without this infrastructure, the EFU remains conceptual.

With it, the EFU becomes **operational, measurable, and globally accessible.**

Document Status: COMPLETE – v1.0
Integration: EFU Master Architecture
Version: 1.0
Date: 2026-03-06

License: EFU Open Research License

FIRE CHIEF MOTTO

“Code is what makes theory real.
The EFU is not philosophy – it is infrastructure.”

Röviden: **az eredeti dokumentum a fő dokumentum**, amit most készítettünk pedig **kiegészítő publikációs formátumok**.
Tehát **nem kell mindent beillesztened az eredetibe**.

Én ezt a struktúrát javaslom, mert így lesz **tiszta, professzionális dokumentációs rendszered**.

Javasolt végső struktúra

1 Fő dokumentum (Word / PDF)

Ez marad az **eredeti, amit most formáztunk**:

200.5 EFU SYSTEM SOFTWARE SPECIFICATION v1.0

Ez legyen a **hivatalos rendszer-specifikáció**.

Tartalmazza:

- architektúra
- adatmodell
- API
- dashboard
- deployment
- roadmap

Ez **teljes és publikálható**.

Tehát:

✔ EZ A FŐ DOKUMENTUM
✔ Ezt publikálhatod
✔ Ez a hivatalos verzió

---

2 Kiegészítő dokumentumok

Amit most készítettem, az **külön fájlokba való**.

A)

EFU Software Architecture – White Paper

Felhasználás:

- Medium
- Substack
- WordPress
- ResearchGate
- LinkedIn

Ez egy **rövidebb, magyarázó verzió**.

B)

GitHub README

Felhasználás:

- GitHub repository
- fejlesztők
- open-source közösség

C)

Architecture diagram

Felhasználás:

- konferencia
- előadás
- pitch deck

Így fog kinézni a teljes EFU rendszer dokumentációja

```
EFU Documentation
│
├── 200.5 EFU System Software Specification (MAIN)
│
├── 205.1 Software Specification Draft
├── 205.2 Data Model
├── 205.3 JSON Schema
├── 205.4 API Documentation
│
├── EFU Software Architecture White Paper
│
├── GitHub Repository
│
└── Conference Architecture Diagram
```

Important: the original document is NOW complete

The 200.5 document is complete:

✔ system architecture

✔ data model

✔ API

✔ dashboard

✔ deployment

✔ roadmap

This is a startup + research level system specification.